

  	ESCUELA DE INGENIERÍA DE SISTEMAS E INFORMÁTICA - ENTORNOS DE PROGRAMACION Creación de Tablas – Excepciones – Validaciones Laboratorio Práctico: API REST de Gestión de Estudiantes	II Semestre 2026
---	--	-------------------------

Laboratorio Práctico: API REST de Gestión de Estudiantes

Objetivo del Ejercicio Construir una API RESTful escalable utilizando el ecosistema de Spring Boot para gestionar el registro académico de estudiantes. El sistema debe garantizar la persistencia de datos relacionales, validación estricta de entradas, control centralizado de errores y exponer su propio contrato a través de documentación interactiva. Se puede entregar con el grupo de trabajo del proyecto.

1. Modelo de Datos y Persistencia

Utilizando Spring Data JPA, se debe mapear la entidad Estudiante para que genere automáticamente la tabla en la base de datos (configurando ddl-auto=update). La entidad debe contener:

- **Cédula:** Llave primaria (@Id).
- **Nombre y Apellidos.**
- **Correo Electrónico.**
- **Dirección de Residencia.**
- **Carrera.**
- **Nota Promedio.**

2. Reglas de Validación (Bean Validation)

Para asegurar la integridad de la información antes de llegar a la capa de servicio, se debe implementar Jakarta Validation en el DTO o Entidad que recibe el controlador:

- **Cédula:** Obligatoria (@NotBlank), contener solo números y tener una longitud de entre 8 y 11 caracteres.
- **Nombre, Apellidos y Carrera:** Campos obligatorios, sin espacios en blanco al inicio o final.
- **Correo Electrónico:** Debe cumplir con un formato de email válido (@Email).
- **Nota Promedio:** Debe ser un valor decimal obligatorio, estrictamente acotado entre 0.0 y 5.0 (@Min, @Max, @NotNull).

  	ESCUELA DE INGENIERÍA DE SISTEMAS E INFORMÁTICA - ENTORNOS DE PROGRAMACION Creación de Tablas – Excepciones – Validaciones Laboratorio Práctico: API REST de Gestión de Estudiantes	II Semestre 2026
---	--	-------------------------

3. Manejo Global de Excepciones

Se requiere implementar un componente anotado con `@RestControllerAdvice` para capturar excepciones y evitar que el cliente reciba el stacktrace genérico de Spring:

- **Errores de Validación:** Capturar `MethodArgumentNotValidException`. La respuesta debe ser un HTTP 400 (Bad Request) que devuelva un mapa o lista JSON detallando exactamente qué campos fallaron y el mensaje de error correspondiente (ej. "correo": "El formato del email es inválido").
- **Errores de Negocio:** Crear una excepción personalizada `EstudianteNoEncontradoException`. Al capturarla, la API debe retornar un HTTP 404 (Not Found) con un mensaje claro.

4. Documentación con OpenAPI (Swagger)

Integrar la dependencia `springdoc-openapi-starter-webmvc-ui` para generar la interfaz gráfica de pruebas.

- Agrupar los endpoints bajo un `@Tag` descriptivo (ej. "Controlador de Estudiantes").
- Describir la funcionalidad de cada endpoint utilizando `@Operation`.
- Documentar las posibles respuestas HTTP utilizando `@ApiResponse`s, especificando qué significa un código 200 OK, 201 Created, 400 Bad Request (avisando explícitamente que ocurre por fallos de validación) y 404 Not Found.

Entregables Esperados

1. Código fuente estructurado en capas (Controller, Service, Repository, Entity/DTO).
2. Archivo `application.properties` con la conexión a la base de datos.
3. Capturas de pantalla de Swagger UI demostrando el registro exitoso de un estudiante y el rechazo de una petición que incumpla las validaciones.